

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Security, SELinux, Firewalls, and Logging

Unit 11 • Linux (10-151-105)



Learning Outcomes

- Harden a Linux system: enforce least privilege, restrict shells, manage banners, and protect data integrity.
- Configure authentication, including Pluggable Authentication Modules (PAM) and multifactor concepts.
- Monitor and audit the system with rsyslog, journalctl, auditd, and logrotate, and recognize compliance standards.
- Apply cryptographic concepts: symmetric and asymmetric encryption, hashing, certificates, and PKI.
- Implement mandatory access control with SELinux (Rocky) and AppArmor (Ubuntu).
- Configure host firewalls — firewalld and zones on Rocky, ufw and nftables on Ubuntu.

FOCUS THIS WEEK

Two topics get extra depth

PAM controls how every login and password change is decided; SELinux policy types control how strictly the whole machine is locked down. Both are flagged clearly as we reach them — give them extra time.

What We'll Cover

Lesson	Focus
1 • Hardening & the Security Mindset	CIA triad, least privilege, restricted shells, banners, data integrity, secure destruction
2 • Authentication & PAM	IAM, MFA, centralized auth, and the PAM framework (focus)
3 • Logging, Auditing & Compliance	rsyslog, journalctl, auditd, logrotate, compliance standards, vulnerability scanning
4 • Encryption & Certificates	Symmetric/asymmetric, hashing, GPG/LUKS/WireGuard, PKI and TLS
5 • Mandatory Access Control	SELinux contexts, modes, policy types (focus); AppArmor on Ubuntu
6 • Host Firewalls	firewalld and zones (Rocky); ufw and nftables (Ubuntu); Fail2Ban

LESSON 1

Hardening & the Security Mindset



The CIA Triad and Least Privilege

Goal	What it protects	Achieved with
Confidentiality	Only authorized users can read the data	Encryption, access controls, authentication
Integrity	Data hasn't changed unexpectedly	Hashing, file-integrity tools, permissions
Availability	Resources are reachable when needed	Backups, redundancy, monitoring, capacity

MINDSET

Least privilege: give each user and service the minimum access required — no more. It limits the blast radius when something is compromised.

Security people say "mitigate," not "eliminate." There is no absolute security — only risk reduced through layered defenses.

Shrinking the Attack Surface

- **Remove what you don't need.**
 - Uninstall unused packages (dnf / apt); disable or mask services with systemctl; never run insecure protocols (Telnet, FTP, TFTP) that send credentials in clear text.
- **Stay current.**
 - Patch the OS and applications promptly and automate it — updates are the single highest-impact routine control.
- **Enforce least privilege.**
 - Set a sensible default umask, tighten permissions and ACLs, and use SELinux/AppArmor for context-based control.
- **Lock down access.**
 - Configure the host firewall, harden SSH, and give service accounts a non-interactive shell.

Restricted Shells and Login Banners

Restricted shell	Purpose
<code>/sbin/nologin</code>	Not a real shell — blocks interactive login. Assign to service/system accounts: <code>useradd -s /sbin/nologin svc</code>
<code>/bin/rbash</code>	"Restricted Bash" — a limited shell for kiosks, test systems, or sandboxed users via a controlled PATH

Login banners — legal notices and system identification:

File	When it shows
<code>/etc/issue</code>	Before a local console login
<code>/etc/issue.net</code>	Before a remote (e.g. SSH) login
<code>/etc/motd</code>	After a successful login — the "message of the day"

Data Integrity and Secure Destruction

Catching intruders — host-based intrusion detection:

AIDE builds a database of file hashes/attributes and reports deviations. **rkhunter** focuses on rootkits and backdoors. Run both for fuller coverage.

Tool	Securely destroys
shred	An individual file (or partition): <code>shred -u -v -n 3 -z secret.txt</code> overwrites 3× plus a zero pass
dd	A whole partition by writing <code>/dev/urandom</code> or <code>/dev/zero</code> — same unforgiving syntax as Unit 5
badblocks	Every block on a device via its write-test mode (<code>-w</code>)

ALSO

Cryptographic destruction ("crypto-shredding"): encrypt the data, then securely destroy the key — the ciphertext becomes permanently unreadable.

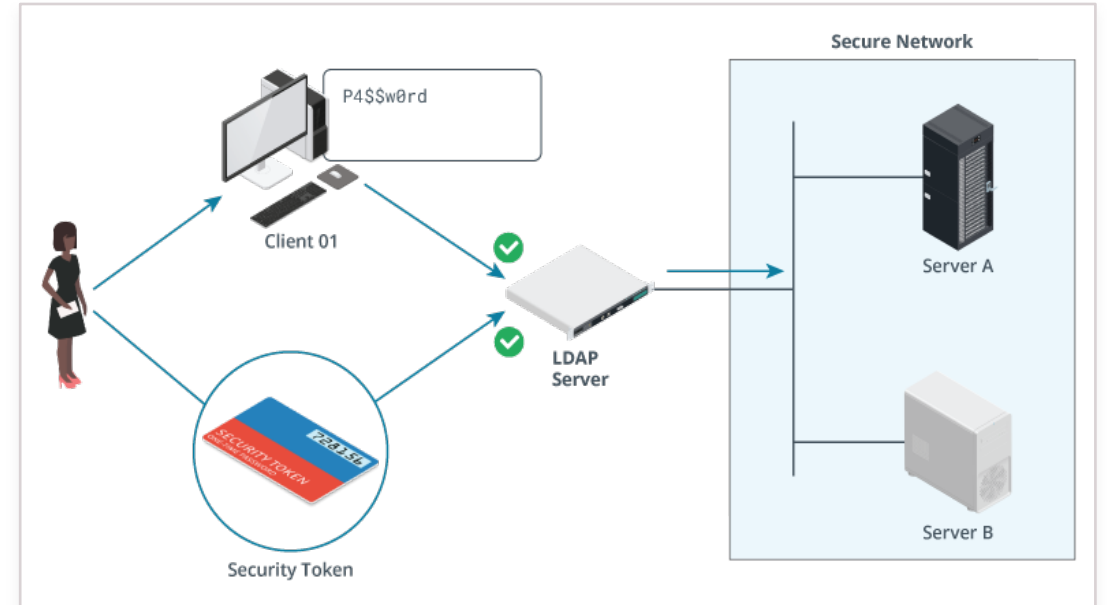
LESSON 2

Authentication & PAM



Authentication and Multifactor (MFA)

- Standard authentication: a username and password, hashed and checked against /etc/shadow.
- Tokens generate one-time passwords (OTP) — hardware (key fob) or software (authenticator app).
- **MFA factors — at least two different kinds:**
 - Something you know (password) · something you have (token) · something you are (biometric).



KEY DISTINCTION

MFA means more than one factor, not more than one method. A hardware token plus a software token is still one factor ("something you have"). A password plus a token qualifies.

Centralized Authentication

SSSD (System Security Services Daemon) connects the local system to central identity providers and caches credentials so logins work offline.

Service	Role
LDAP / LDAPS	Directory protocol for account data; LDAPS wraps it in SSL/TLS to protect the directory traffic
Kerberos	Ticket-based authentication with SSO; integrates Linux with Active Directory. Provides authentication only — authorization still comes from Linux permissions and ACLs
Samba	Implements Windows SMB so Linux can host or access Windows file shares; has local accounts and can check credentials via LDAP

Pluggable Authentication Modules (PAM)

FOCUS TOPIC

PAM decides every login on the system

Once you understand PAM's four module interfaces and four control flags, a huge amount of Linux security configuration — password rules, lockouts, login restrictions — stops being mysterious.

- PAM provides one common framework that login, sshd, sudo, passwd, and others all use to decide "should this succeed?"
- This is also how Kerberos and LDAP integrate — they plug in as PAM modules.
- Configuration lives in `/etc/pam.d/`, with one file per service (sshd, sudo, passwd, ...).

Reading a PAM Directive

```
# module-interface    control-flag    module-name    [arguments]
password              requisite      pam_pwquality.so    retry=3
```

Part	In the example	Meaning
Module interface	password	This rule applies when a password is being changed
Control flag	requisite	If this module fails, stop immediately and tell the user
Module name	pam_pwquality.so	The module that checks password strength
Arguments	retry=3	Give the user three attempts to supply an acceptable password

PAM Module Interfaces

Interface	Handles
auth	Verifying identity — checking the password and establishing credentials (e.g. a Kerberos ticket)
account	Whether a verified user is allowed access right now — account expiry, time-of-day limits
password	Changing the authentication token — where password-quality and history rules live
session	Setting up and tearing down the session — mounting the home directory, logging the login, limits

REMEMBER

Four interfaces — auth, account, password, session. They map to the four phases of letting someone in and getting them set up.

PAM Control Flags

Flag	Behavior
required	Must succeed for auth to pass — but the stack keeps running and the user isn't told until all modules have run (hides which check failed)
requisite	Like required, but failure stops the stack immediately and reports at once
sufficient	If it succeeds (and nothing required failed first), that's enough — the rest is skipped. If it fails, its result is ignored
optional	Its result is ignored for the decision — it matters only if it's the only module of its type

THE COMMON MIX-UP

required vs. requisite: both must succeed. The difference is timing — requisite fails fast and tells the user; required finishes the stack first.

PAM for Password Policy & Lockout

Module	Enforces
pam_pwquality.so	Password strength — length, character classes, dictionary rejection (replaces pam_cracklib.so)
pam_pwhistory.so	Password history — blocks reuse of recent passwords (e.g. remember=24)
pam_faillock.so	Account lockout after repeated failures, with auto-unlock (replaces pam_tally2)

DUAL-DISTRO NOTE

Both use /etc/pam.d/, but the shared-stack filenames differ:
Rocky: system-auth, password-auth
Ubuntu: common-auth, -account, -password, -session

CAUTION

A broken PAM stack can lock everyone out — including you. Keep a root session open in a second terminal while you test a change in a new login.

LESSON 3

Logging, Auditing & Compliance



Where the Logs Are

Log	Rocky (Red Hat)	Ubuntu (Debian)
General system log	/var/log/messages	/var/log/syslog
Authentication log	/var/log/secure	/var/log/auth.log
Boot log	/var/log/boot.log	/var/log/boot.log

DUAL-DISTRO NOTE

When you go hunting for who logged in, remember the family split: Red Hat uses secure / messages; Debian uses auth.log / syslog. Same idea, different filename.

journalctl and rsyslog

Querying the journal for security events:

```
$ journalctl | grep -i "authentication failure"    # failed local auth
$ journalctl -u ssh | grep -i "failed password"    # failed SSH logins
$ last          # recent logins          lastlog    # last login per user
```

rsyslog routes each message by a **facility** (source: cron, kern, auth...) and a **severity** (0 emerg → 7 debug). Rules in /etc/rsyslog.conf map them to destinations; it can also forward logs to a central server on port 514.

```
#cron.*          /var/log/cron.log
#daemon.*        -/var/log/daemon.log
kern.*           -/var/log/kern.log
#lpr.*           -/var/log/lpr.log
mail.*           -/var/log/mail.log
#user.*          -/var/log/user.log
```

Kernel-Level Auditing with auditd

Watch sensitive account files; merge and list the rules:

```
-w /etc/passwd -p wa -k user_changes
-w /etc/shadow -p wa -k password_changes
-w /etc/sudoers -p wa -k sudoers_change
$ augenrules --check # validate syntax
$ augenrules --load # activate rules
$ auditctl -l # list active rules
```

- Search and report with ausearch and aureport — e.g. aureport --failed.
- Works directly with the kernel, so it catches events other tools can't.
- Pairs naturally with CIS benchmarks for compliance monitoring.

logrotate and Compliance Standards

logrotate runs daily to rotate, compress, and delete old logs. Global config: /etc/logrotate.conf; per-app overrides: /etc/logrotate.d/. Key directives: rotate N, compress, size / maxage, dateext, create.

Standard	What it is
OpenSCAP	Open-source framework that scans Linux against a baseline and reports
CIS Benchmarks	Community baseline security configurations per distribution
NIST CSF	Framework: identify, protect, detect, respond, recover
STIGs	U.S. Defense security implementation guides
HIPAA / PCI-DSS	Industry regulations — healthcare and payment-card data

Vulnerability Scanning, CVE & CVSS

Tool	Purpose
Nmap	Port scanner — maps open ports and services
Wireshark / tcpdump	Packet analyzers — capture and read network traffic
OpenVAS	Vulnerability scanner for networks and web apps

- CVE — the public catalog of known vulnerabilities, maintained by MITRE.
- CVSS — a 0–10 severity score that helps you prioritize which to fix first.

CAUTION

Running scanners like Nmap or Wireshark in production can trip intrusion-detection systems and may violate acceptable-use policy. Only scan systems you're explicitly authorized to test.

LESSON 4

Encryption & Certificates



Encryption Concepts

- A cipher (algorithm) plus a key converts plaintext to ciphertext; only the right key reverses it.
- Protects data in three states: in transit, in use (memory), and at rest (disk).
- Encryption goals: confidentiality, integrity, and non-repudiation. Not all encryption aims at secrecy.

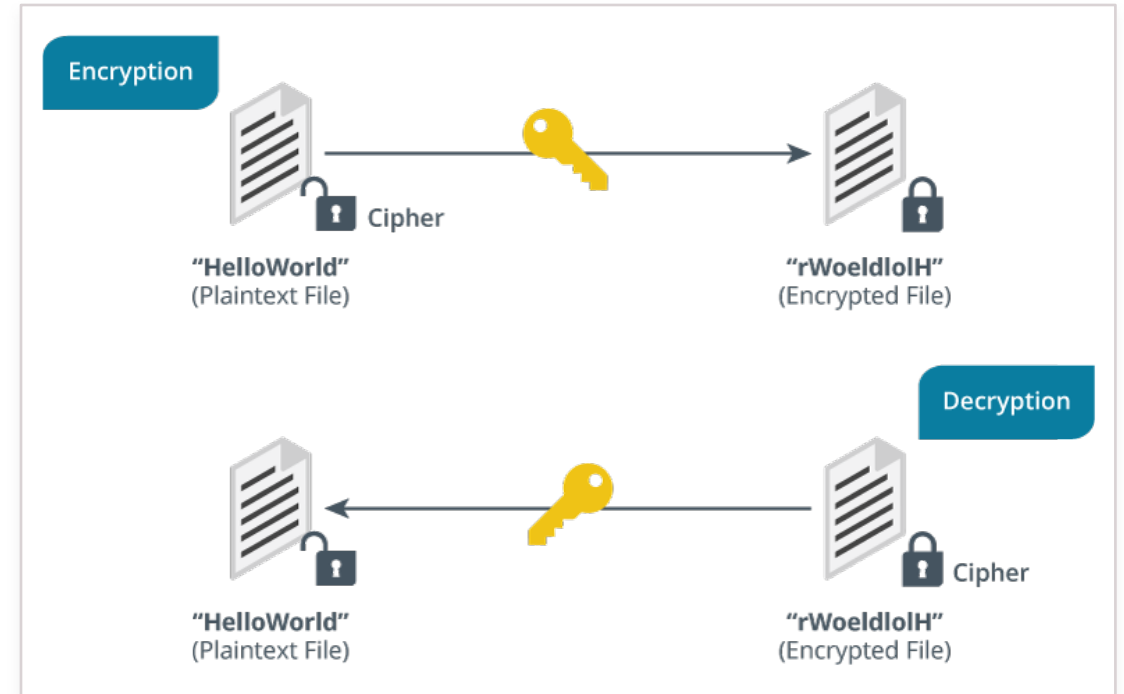


Figure: encryption converts plaintext to ciphertext using a cipher and a key; decryption reverses it.

Symmetric vs. Asymmetric

	Symmetric	Asymmetric
Keys	One shared key encrypts and decrypts	A key pair — a public key and a private key
Speed	Fast — light on the CPU	Slower — heavier math
Main challenge	Sharing the key safely	None for key delivery — the public key is shareable
Good for	Bulk data, files, fast transfers	Key exchange, digital signatures, e-commerce

IN PRACTICE

The two are combined: encrypt bulk data fast with a symmetric key, then protect that key by encrypting it with the recipient's public key. You get symmetric speed and safe key delivery.

Public-Key vs. Private-Key Encryption

Use	Encrypt with	Decrypt with	Achieves
Public-key encryption	Recipient's public key	Recipient's private key	Confidentiality — only the recipient can read it (e-commerce)
Private-key encryption	Your own private key	Your public key (anyone)	Non-repudiation — proves you sent it (a digital signature)

THE TEST QUESTION

"To send a confidential message only Bob can read, encrypt with..." → Bob's public key. (Encrypting with your own private key would be a signature, not secrecy.)

Hashing and Integrity

- Publish a file and its hash; anyone can re-hash their copy and confirm it matches.
- Hashing reveals that data changed — not what changed.
- SHA-256 is the modern standard — used in passwords, signatures, blockchain, and TLS.

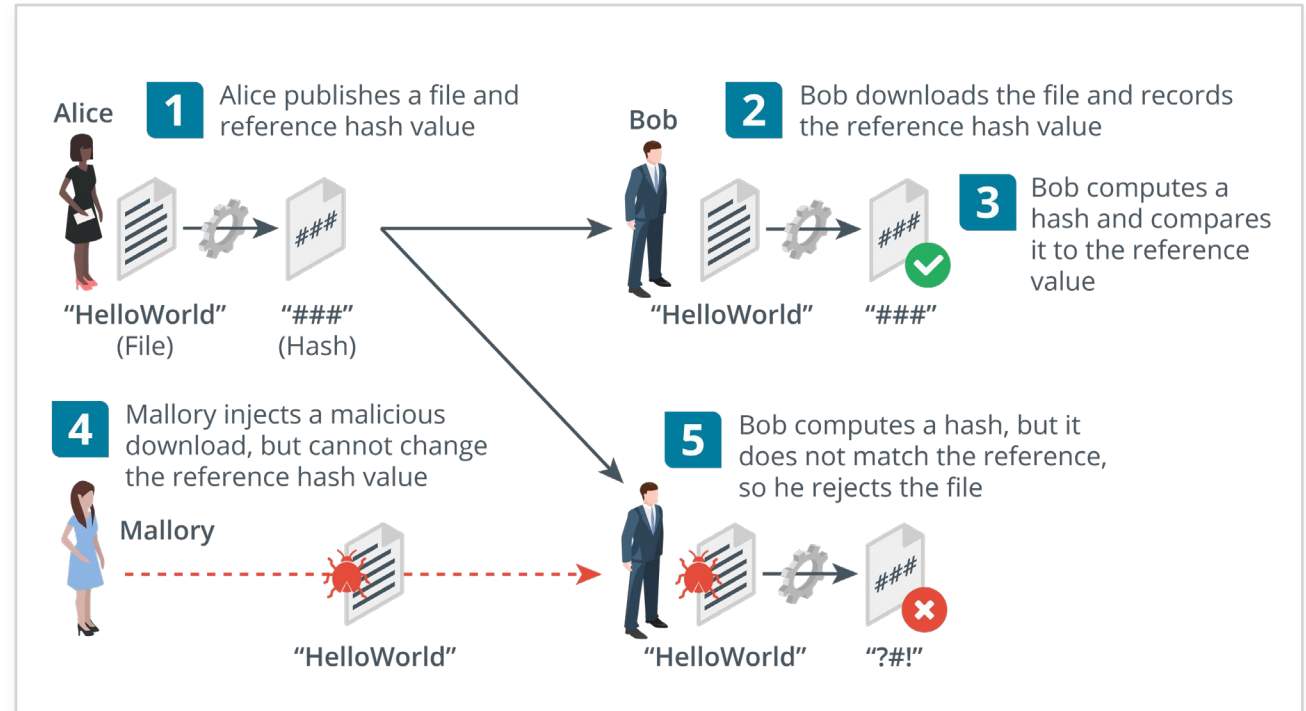


Figure: Alice publishes a file and its hash; Bob re-hashes his copy. A match means intact; when Mallory tampers, the hash differs and Bob rejects it.

Protecting Data at Rest and in Transit

Tool	Protects	Notes
GPG	Files at rest	Combines symmetric speed with asymmetric key protection. <code>gpg -c file</code> encrypts; <code>gpg -d</code> decrypts
LUKS	Whole disks/partitions at rest	Full-disk encryption for an entire drive, partition, or volume
WireGuard	Data in transit	Modern VPN built into the Linux kernel — faster and simpler than IPsec/OpenVPN

Certificates and PKI

- A digital certificate binds a public key to an identity, with issuer, holder, validity dates, and the issuer's signature.
- PKI manages the lifecycle: request → issue → use → expire → revoke.
- **A CA signs certificates so clients can validate them:**
 - Third-party CA — trusted publicly (customer ↔ store). Internal CA — self-signed, internal only.
 - Wildcard cert covers all subdomains of one domain.
- TLS uses certificates to secure web traffic (the HTTPS in a URL); OpenSSL manages keys and certs.



Certifying Root CA 9909
Root certificate authority
Expires: Monday, May 14, 2029 at 2:25:42 PM Mountain Daylight Time
✔ This certificate is valid

▶ Trust
▼ Details

Subject Name	
Organization	Certifying Company
Common Name	Certifying Root CA 9909
Issuer Name	
Organization	Certifying Company
Common Name	Certifying Root CA 9909
Serial Number	5F F8 7B 28 2B 54 DC 8D 42 A3 15 B5 68 C9 AD FF
Version	3
Signature Algorithm	SHA-1 with RSA Encryption (1.2.840.113549.1.1.5)
Parameters	None
Not Valid Before	Friday, May 14, 2004 at 2:17:12 PM Mountain Daylight Time
Not Valid After	Monday, May 14, 2029 at 2:25:42 PM Mountain Daylight Time
Public Key Info	
Algorithm	RSA Encryption (1.2.840.113549.1.1.1)
Parameters	None
Public Key	256 bytes: B0 9A B9 AB A7 AF 0A 77 ...

NEVER

Never use a self-signed certificate for a public-facing site — browsers warn users and it isn't compliant. Self-signed is for internal/test use only.

LESSON 5

Mandatory Access Control



DAC vs. MAC, and SELinux

FOCUS TOPIC

SELinux is the hardest idea in the unit

It adds a second, stricter layer on top of normal permissions. The single most important configuration choice is the policy type — flagged shortly.

- DAC (discretionary): the file owner decides access — and a careless user or hijacked program can loosen it.
- MAC (mandatory): a system-wide policy only admins can change, enforced on every process.
- SELinux (Security-Enhanced Linux), from the NSA, provides MAC on Rocky/Red Hat. Even if permissions would let Apache read a user's files, policy can forbid it.

SELinux Contexts (Labels)

Field	Meaning
User	An SELinux user each Linux account maps to (unconfined_u, user_u, sysadm_u)
Role	Controls which domains a user may enter; files use object_r
Type	The key field for fine-grained control. A type on a process is a domain (e.g. ssh_t)

- ls -Z file — show a file's context
- ps -Z — show a process's context
- chcon -t ... file — change a context (temporary)
- restorecon -R dir — reset to the default (safer)

```
[student@fedora etc]$ ls -Z ~
unconfined_u:object_r:user_home_t:s0 Desktop
unconfined_u:object_r:user_home_t:s0 Documents
unconfined_u:object_r:user_home_t:s0 Downloads
unconfined_u:object_r:audio_home_t:s0 Music
unconfined_u:object_r:user_home_t:s0 myfile.txt
unconfined_u:object_r:user_home_t:s0 Pictures
unconfined_u:object_r:user_home_t:s0 Public
unconfined_u:object_r:user_home_t:s0 Templates
unconfined_u:object_r:user_home_t:s0 Videos
```


SELinux Modes

Mode	Behavior
Enforcing	Policy is active; violations are blocked and logged. The normal protected state
Permissive	Policy is active but violations are logged, NOT blocked. The troubleshooting state
Disabled	SELinux is off entirely. Not recommended

```
$ getenforce          # current mode
$ sestatus            # fuller status
$ setenforce 0        # → permissive
$ setenforce 1        # → enforcing
```

```
[student@fedora etc]$ getenforce
Enforcing
[student@fedora etc]$ sudo setenforce 0
[student@fedora etc]$ getenforce
Permissive
[student@fedora etc]$ sudo setenforce 1
[student@fedora etc]$ getenforce
Enforcing
```

SELinux Policy Types

FOCUS — POLICY TYPES

An SELinux policy defines access rules for every process and resource. Which policy TYPE is loaded determines how much of the system is actually confined.

Policy type	What it does
Targeted	THE DEFAULT. Only specific daemons (httpd, mariadb...) run confined under MAC; everything else runs unconfined under DAC. Protects high-risk services without confining the whole system
Strict	The opposite — every subject and object is forced under MAC. Maximum control, far more complex. For high-security environments
Minimum	Like targeted but confines even fewer processes and loads less policy into memory. For small/embedded devices or experimentation

sestatus shows which policy type is loaded; policy files live in /etc/selinux/ (root only).

SELinux Booleans & Troubleshooting

```
$ getsebool -a                # list Booleans
$ setsebool -P httpd_can_network_connect on    # persistent
$ sealert -a /var/log/audit/audit.log          # diagnose denials
```

- Booleans are on/off switches for policy options — no recompiling. -P makes the change survive reboot.
- sealert explains a denial and suggests a fix; audit2allow turns a legitimate denial into an allow rule; restorecon fixes a wrong label.

CLASSIC GOTCHA — COPY VS. MOVE

mv keeps a file's original context — an HTML file moved into /var/www/html still carries the home-dir type, so Apache can't read it. cp inherits the destination's context, so it works. Fix a moved file with restorecon.

AppArmor (Ubuntu)

	SELinux (Rocky)	AppArmor (Ubuntu)
Controls by	Inode labels — types & domains	File paths — no types/domains
Configuration	Compiled policy + contexts; granular	Flat per-program profiles; simpler
Modes	Enforcing / permissive / disabled	enforce / complain

- Profiles live in /etc/apparmor.d/, named for the binary path; they list capabilities and path entries.
- apparmor_status (loaded profiles & modes) · aa-complain / aa-enforce / aa-disable · aa-unconfined (network apps with no profile).

```
student@ubuntu20:~$ sudo apparmor_status
[sudo] password for student:
apparmor module is loaded.
37 profiles are loaded.
35 profiles are in enforce mode.
  /snap/snapd/11036/usr/lib/snapd/snap-confine
  /snap/snapd/11036/usr/lib/snapd/snap-confine/
  /snap/snapd/15177/usr/lib/snapd/snap-confine
```

LESSON 6

Host Firewalls



Host Firewalls: Stateful Filtering

	Rocky (Red Hat)	Ubuntu (Debian)
Front-end	firewalld (firewall-cmd)	ufw, with nftables underneath
Organizing idea	Zones — trust levels bound to interfaces	A simple ordered rule list
Default in the pod	firewalld enabled	ufw installed but inactive — you enable it

STATEFUL VS. STATELESS

A stateful firewall tracks connection state (new, established, related) and automatically allows return traffic for connections you started — rather than evaluating each packet in isolation as a stateless filter would.

firewalld and Zones (Rocky)

```
# firewall-cmd --get-active-zones    # which zones are in use
# firewall-cmd --zone=public --add-service=https --permanent
# firewall-cmd --reload             # apply permanent rules
# firewall-cmd --zone=public --list-all    # review the zone
```

RUNTIME VS. PERSISTENT

Without `--permanent`, a rule is lost on reload or reboot — the same runtime-vs-persistent distinction you saw with SELinux Booleans (-P).

ufw, nftables, and Fail2Ban (Ubuntu)

```
# ufw default deny incoming    # least-privilege baseline
# ufw allow OpenSSH           # allow SSH by service name
# ufw enable                  # turn the firewall on
# ufw status verbose          # review the rules
```

FAIL2BAN

Fail2Ban watches auth logs for repeated failures (e.g. SSH brute-force) and dynamically bans the offending IP via a firewall rule — tying this unit's logging to its firewall.

CAUTION

Set default-deny AFTER allowing SSH, or you'll lock yourself out of a remote session. In the pod, work from the console so a mistake can't strand you.

The Lab: Security, SELinux, Firewalls & Logging

- Rocky — set SELinux modes and resolve a denial with the correct context or Boolean.
- Ubuntu — manage an AppArmor profile's state (complain ↔ enforce).
- Both — build host firewall rules that expose only the required services (firewalld vs. ufw).
- Both — investigate a security-relevant event in the logs.
- **Framing: open only what is needed, and confirm the rest is closed.**

REMEMBER

Your work doesn't persist between reservations — the pod reverts to its WK11_SECURITY snapshot. Capture your verification evidence before you log out.

Key Takeaways



What to carry out of Unit 11

- Hardening is a posture: remove what you don't need, patch what remains, grant least privilege.
- PAM decides every login — know the four interfaces (auth/account/password/session) and four control flags.
- Logging and auditing make a system investigable: rsyslog, journalctl, auditd, logrotate.
- Encryption: symmetric for speed, asymmetric for key exchange and signatures; hashing proves integrity.
- MAC adds a stricter layer: SELinux (contexts, modes, policy types) on Rocky; AppArmor on Ubuntu.
- Host firewalls enforce least privilege on the network: firewalld/zones on Rocky, ufw/nftables on Ubuntu.

Why it matters: "working" and "secure" are different bars — both must be met.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 11 · Security, SELinux, Firewalls, and Logging

